

Verilog Language and Application

Week 9 Lab Document

Lab Manual

Revision 1.0

© 1990–2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;

- The publication may not be modified in any way;

- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

Verilog Language and Application

Lab 1	Testing the VeriRISC CPU Model.....	5
	Specifications.....	5
	Designing the Testbench for VeriRISC CPU Verification.....	6
	Verifying VeriRISC CPU Design	7

Lab 1 Testing the VeriRISC CPU Model

Objective: To interactively select and download test microcode and run it.

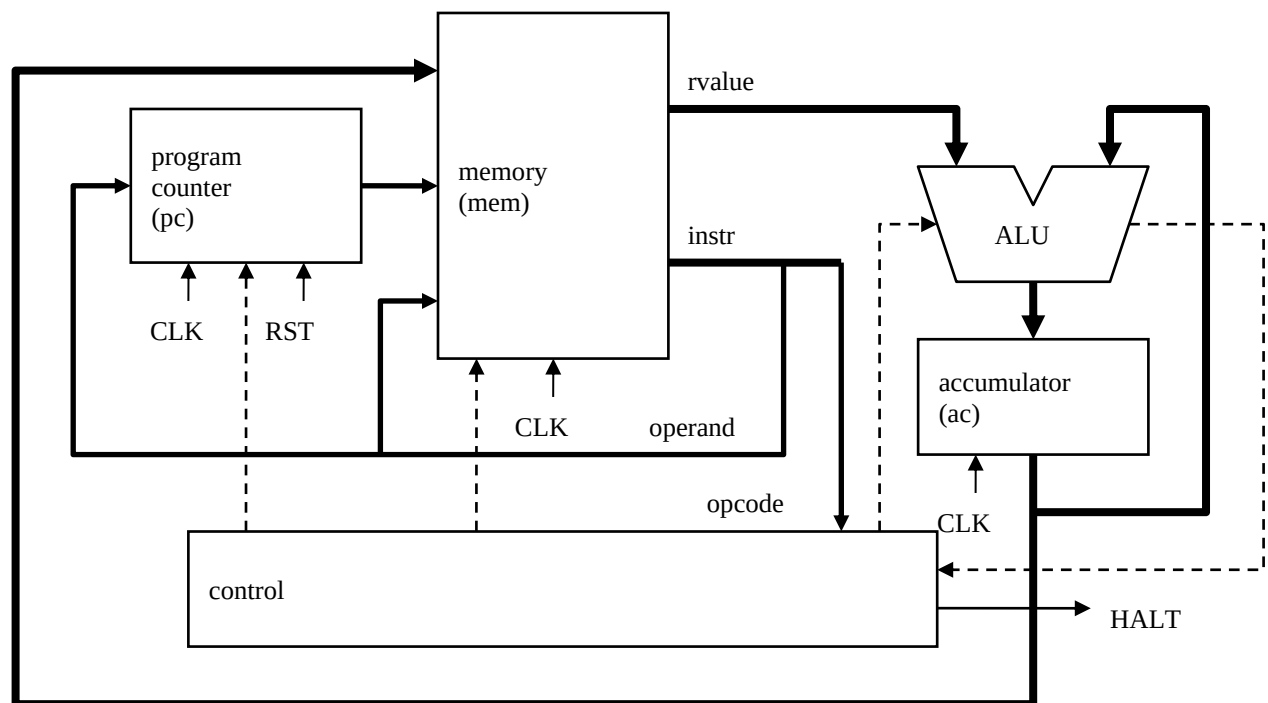
The lab model is a VeriRISC CPU. In the Lab-12 specification, the CPU architecture and operation is explained. In this specification, a few more operations are explained.

Read the specification first and then follow the instructions in the lab.

Specifications

The CPU architecture is as follows:

- ◆ The CPU operates on the positive clock edge.
- ◆ It has a synchronous high-active reset.
- ◆ The CPU uses a two-port memory so that it can fetch and execute an instruction on each clock cycle.
- ◆ The 8-bit CPU instruction consists of a 3-bit leftmost operation code encoding eight instructions.
- ◆ And a 5-bit rightmost operand addressing up to 32 words.



Designing the Testbench for VeriRISC CPU Verification

1. In the previous block diagram, you can observe the CPU has only HALT as output.
2. The testbench environment can detect program failure only by detecting that the CPU halted at an incorrect address.
3. This lab provides three programs with incremental levels of complexity that halt (when operating correctly) at different program counter values:

For program 1 halt value is – 0x17

For program 2 halt value is – 0x10

For program 3 halt value is – 0x0c

4. You have to code the specified testbench, which can have well-defined tasks for the smoother execution of the programs.
 - a. First, code a task displaying an initial message informing the user what commands to use to operate the test. You can write the task by using the following interactive command in which you have to place a value in a test register.

deposit <i>object_name value</i>	Deposit a value onto a simulation object
task <i>task_name</i>	Schedule a Verilog task for immediate execution
run <i>time_spec</i>	Continue the simulation to the next breakpoint, interrupt, or (optional) time point

Note: For example: Deposit a value to a register/variable inside a task like {deposit *module_name.task_name.variable_name value*} **deposit test.run.number 1** where “number” is the register/variable.

5. Code a task to run a program in which the task creates a filename (which is a string) by concatenating the value of a register/variable that the user sets by using the *deposit* command. In the lab directory, you can see the txt filename convention that you use to name the programs such as *PROGn.txt*.

Note: The ASCII value of the 0 character is 0x30 so your concatenation would be something like {"PROG", 8'h30 + *test_number_reg*, ".txt "}

6. Load that program file into the CPU memory. Reset the CPU. To avoid a clock/data race, you can move the reset signal on the opposite clock edge in which the CPU works (use **negedge**). After reset, the CPU automatically starts executing at location 0.

7. Code a task displaying a final message informing the user of the program counter address where the CPU halted and whether that is correct.
8. Code a procedural block generating a free-running clock.
9. Code a procedural block that upon startup:
 - a. Displays the initial message.
 - b. Pauses the simulation (use **\$stop**).
10. Code a procedural block that upon every assertion of the HALT signal:
 - a. Displays the final message.
 - b. Again, displays the initial message.
 - c. Again, pauses the simulation.

Verifying VeriRISC CPU Design

1. Change to the *lab1-cpu* directory and examine the following files.

cpu.v	CPU module
PROG1.txt PROG2.txt PROG3.txt	Program files

2. Code the specified test with well-defined tasks for the smoother execution of the programs.
3. Simulate the design and test using the following commands with Xcelium.

```
xrun cpu.v your_test_file_name -access +rwc
```

4. Execute the following commands and compare the outputs after each run.

```
xcelium> deposit test.run.number 1; task test.run; run
Halted at address = 17
Expected address = 17
TEST PASSED
```

Testing the VeriRISC CPU Model

```
xcelium> deposit test.run.number 2; task test.run; run
```

```
Halted at address = 10
```

```
Expected address = 10
```

```
TEST PASSED
```

5. Correct your test and repeat as needed.

